Εθνικο & Καποδιστριακο Πανεπιστημιο Αθηνων
Τμημα Πληροφορικης & Τηλεπικοινωνιων
Μαθημα: **Αρχιτεκτονικη Υπολογιστων ΙΙ**

Εργασία 1 (*υποχρεωτική*) – Διοχέτευση

Ακαδημαϊκο Ετος 2024 – 2025
(εκφωνηση) Παρασκευη 15 Νοεμβριου 2024
(παραδοση στο eclass μεχρι) **Τριτη 3 Δεκεμβριου 2024**

Επώνυμο – Όνομα	Πατρώνυμο – Μητρώνυμο	Αριθμός Μητρώου	Email
Καραπροδρομίδης Ιάσονας	-	1115202200064	iaskarapro@gmail.com
Ζορμπά Αλεξάνδρα	-	1115202200048	alexandra.n.zorba@gmail.com

Πληροφορίες για τις Υποχρεωτικές Εργασίες του μαθήματος___

Οι υποχρεωτικές εργασίες του μαθήματος είναι **δύο**. Σκοπός τους είναι η κατανόηση των εννοιών του μαθήματος με χρήση αρχιτεκτονικών προσομοιωτών. Η πρώτη υποχρεωτική εργασία (αυτή) αφορά τη διοχέτευση (pipelining) και η δεύτερη θα αφορά τις κρυφές μνήμες (cache memories).

- Οι δύο εργασίες είναι υποχρεωτικές και η βαθμολογία του μαθήματος θα προκύπτει από το γραπτό (60%), την εργασία της διοχέτευσης (20%), και την εργασία των κρυφών μνημών (20%).
- Κάθε ομάδα μπορεί να αποτελείται **από 1 έως και 2 φοιτητές**. Συμπληρώστε τα στοιχεία των μελών της ομάδας στον παραπάνω πίνακα. Τα μέλη της ομάδας πρέπει να έχουν ισότιμη συμμετοχή και να γνωρίζουν τις λεπτομέρειες της υλοποίησης της ομάδας.
- Για την εξεταστική Σεπτεμβρίου δε θα δοθούν άλλες εργασίες. Τον Σεπτέμβριο εξετάζεται μόνο το γραπτό.
- Σε περίπτωση αντιγραφής θα μηδενίζονται όλες οι ομάδες που μετέχουν σε αυτή.
- Η παράδοση της **Εργασίας Διοχέτευσης** πρέπει να γίνει μέχρι τα **μεσάνυχτα της προθεσμίας ηλεκτρονικά** και **μόνο στο eclass από ένα μόνο μέλος της ομάδας** (να ανεβάσετε ένα μόνο αρχείο zip ή rar με την τεκμηρίωσή σας σε PDF και τον κώδικά σας). **Μην περιμένετε μέχρι την τελευταία στιγμή. Η εκφώνηση της εργασίας 2 των κρυφών μνημών θα αναρτηθεί αμέσως**.

Ζητούμενο

Υποθέστε ότι λειτουργείτε ένα κέντρο δεδομένων με 10 000 MIPS CPUs και σας ενδιαφέρει το **συνολικό κόστος ιδιοκτησίας (total cost of ownership – TCO)** του κέντρου δεδομένων. Το TCO για ένα ημερολογιακό έτος ορίζεται απλουστευτικά για τις ανάγκες της άσκησης ως: **κόστος αγοράς των τσιπ των CPU + δαπάνη ενέργειας για την ετήσια λειτουργία όλων των CPU**. Κάθε 12 μήνες αγοράζετε καινούριες CPU και ανακυκλώνετε τις παλιές. Ως ετήσια **υπολογιστική απόδοση** για το κέντρο δεδομένων ορίζεται το πλήθος των πλήρων εκτελέσεων του ενός και μοναδικού προγράμματος που εκτελούν συνεχώς όλες οι CPU στο κέντρο δεδομένων στη διάρκεια ενός έτους. Το ζητούμενο είναι να βελτιστοποιήσετε (μεγιστοποιήσετε) τον λόγο:

Υπολογιστική Απόδοση προς Συνολικό Κόστος Ιδιοκτησίας

Το πρόγραμμα που εκτελούν συνεχώς για 24 ώρες κάθε μέρα όλες οι CPU στο κέντρο δεδομένων είναι ένα πρόγραμμα (που πρέπει να γράψετε σε assembly MIPS) το οποίο αναζητά σε έναν μη ταξινομημένο πίνακα 200 προσημασμένων ακεραίων αριθμών A[i] (i = 0, 1, ... 199) την εξής πληροφορία: (α) πόσοι από τους αριθμούς του πίνακα είναι αρνητικοί (N), (β) πόσοι είναι θετικοί (P), (γ) πόσοι είναι ίσοι με το μηδέν (Z), (δ) πόσοι είναι άρτιοι (E), (ε) πόσοι είναι περιττοί (O). Τα A, N, P, Z, E, O να είναι μεταβλητές στο τμήμα δεδομένων του προγράμματός σας. Δεν πρέπει να δίνονται αριθμοί από την κονσόλα, ούτε και να εκτυπώνεται τίποτε σε αυτή.

Οι εναλλακτικές σχεδιάσεις CPU MIPS που εξετάζετε είναι:

(α) MIPSa: επεξεργαστής διαδρομής δεδομένων ενός κύκλου ρολογιού με ρολόι 50 MHz, κόστος 5 ευρώ, και ηλεκτρική ισχύ 20 watt

(β) MIPSb: επεξεργαστής με διοχέτευση πέντε σταδίων αλλά χωρίς προώθηση (μόνο ανίχνευση κινδύνων δεδομένων και προσθήκη καθυστερήσεων για τη σωστή εκτέλεση) και με προσέγγιση καθυστέρησης σε διακλάδωση (stall on branch) και επίλυση διακλαδώσεων στο στάδιο EX – το ρολόι του είναι 150 MHz, κοστίζει 8 ευρώ, και δαπανά ισχύ 35 watt

(γ) MIPSc: επεξεργαστής με διοχέτευση πέντε σταδίων, πλήρη προώθηση, και πρόβλεψη διακλάδωσης με 2-bit, BHT των 5 bit, και επίλυση στο στάδιο ID – το ρολόι του είναι 150 MHz, κοστίζει 12 ευρώ, και δαπανά ισχύ 45 watt

Το κόστος της ενέργειας ανά kWh (κιλοβατώρα) είναι 0.002 euro.

Ποια από τις τρεις CPU θα δώσει για το πρόγραμμα που γράψατε τον καλύτερο λόγο: Υπολογιστική Απόδοση προς Συνολικό Κόστος Ιδιοκτησίας;

Μας ζητήθηκε να φτιάξουμε ένα πρόγραμμα το οποίο υπολογίζει το πλήθος θετικών, αρνητικών, άρτιων και περιττών αριθμών σε έναν μη ταξινομημένο πίνακα 200 θέσεων σε πρόγραμμα χωρίς είσοδο ή έξοδο.

Ξεκινώντας την υλοποίηση της εργασίας φτιάξαμε ένα απλό, αλλά ορθό, πρόγραμμα που έκανε διακλάδωση για κάθε είδος αριθμού. Αυτό το πρόγραμμα είχε την εξής επίδοση:

MipsA

Cycle Statistics

Total Cycles:	2658
Instructions:	2658
CPI:	1
Data Hazard Stalls:	0
Control Hazard Stalls:	0
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

MipsB

Cycle Statistics

Total Cycles:	5161
Instructions:	2661
CPI:	1.9395
Data Hazard Stalls:	426
Control Hazard Stalls:	2074
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

MipsC

Cycle Statistics

Total Cycles:	3095
Instructions:	2661
CPI:	1.1631
Data Hazard Stalls:	200
Control Hazard Stalls:	234
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

Η επίδοση αυτού του προγράμματος ήταν εξαιρετικά χαμηλή με πολλούς κινδύνους ελέγχου και αρκετούς κινδύνους δεδομένων. Η κύρια πηγή των προβλημάτων ήταν το πλήθος των branch instructions που κάναμε, τα οποία οδηγούσαν βεβαίως σε πολλά διαφορετικά tags. Μέρος αυτού του κώδικα που το αναδεικνύει:

```
          bne $t8,$zero,Negative
EvenOdd:
        andi $t9, $t7, 1 # Perform bitwise AND with 1
        beq $t9, $zero, is_even # If result is 0, jump to is_even
        j is_odd
Positive:
        addi $t3,$t3,1
        j EvenOdd
Zero:
        addi $t4,$t4,1
        j EvenOdd
Negative:
        addi $t2,$t2,1
        j EvenOdd
is_even:
        addi $t5,$t5,1
        j check
is odd:
```

Αρχίσαμε τη μείωση των branches με την αντικατάστασή τους από j (jump) εντολές όπου ήταν προφανές (για τους αρνητικούς που δεν είναι ούτε θετικοί, ούτε μηδέν και για τους άρτιους που δεν είναι, προφανώς, περιττοί). Η συγκεκριμένη τροποποίηση άφησε σχεδόν ανεπηρέαστο τον mipsC, καθώς κατέχει 2 bit branch predictor και θα προέβλεπε ανεξαρτήτως όλες τις διακλαδώσεις, εκτός από την πρώτη και την τελευταία.

```
loop:
        lw $t7,0($t0)
        addi $t1,$t1,-1
        addi $t0,$t0,4
        slt $t8,$t7,$zero
        beq $t7,$zero,Zero
        beq $t8,$zero,Positive
        #j Negative
Negative:
        addi $t2,$t2,1
        #j EvenOdd
EvenOdd:
        andi $t9, $t7, 1 # Perform bitwise AND with 1
        beq $t9, $zero, is_even # If result is 0, jump to is_even
        j is_odd
Positive:
        addi $t3,$t3,1
        j EvenOdd
```

Εκμεταλλευόμενοι, επίσης το continuous allocation του data section, μειώσαμε το πλήθος των εντολών $la (load address).

```
#due to continuous allocation of memory

   la $t0, N
   lw $t2,0($t0)   # $t2 = N (αρνητικοί)
   lw $t3,4($t0)   # $t3 = P (θετικοί)
   lw $t4,8($t0)   # $t4 = Z (μηδενικοί)
   lw $t5,12($t0)  # $t5 = E (άρτιοι)
   lw $t6,16($t0)  # $t6 = O (περιττοί)
```

Προχωρώντας, αποφασίσαμε να υπολογίζουμε τα αθροίσματα των αρνητικών και των αρτίων στο τέλος, αφαιρώντας από τους 200 αριθμούς του πίνακα τους θετικούς-μηδενικά για τους αρνητικούς και τους περιττούς για τους αρτίους, αντίστοιχα.

Σε κάθε στάδιο βελτιστοποίησης του προγράμματος, ελαττώναμε τα data hazards με αναδιάταξη εντολών αποφεύγοντας να τοποθετούμε εντολές με εξάρτηση (κοινό register στο οποίο γίνεται read after write, το γνωστό RAW πρόβλημα) τη μία κάτω από την άλλη.

Είχαμε φτάσει σε ένα αξιοσημείωτο για εμάς σημείο (milestone) και είχαμε αυτές τις βελτιωμένες, αλλά βεβαίως όχι και πολύ ικανοποιητικές αποδόσεις:

MipsA

MipsB

Cycle Statistics

Total Cycles:	2421
Instructions:	2421
CPI:	1
Data Hazard Stalls:	0
Control Hazard Stalls:	0
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

Cycle Statistics

Total Cycles:	4548
Instructions:	2424
CPI:	1.87624
Data Hazard Stalls:	347
Control Hazard Stalls:	1777
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

MipsC

Cycle Statistics

Total Cycles:	2811
Instructions:	2424
CPI:	1.15965
Data Hazard Stalls:	169
Control Hazard Stalls:	218
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

Δεν ήμασταν ικανοποιημένοι με τα control hazard stalls, καθώς ήταν ακόμη πάρα πολλά στον mipsB. Για τη μείωσή τους, έπρεπε να μειώσουμε τις εντολές διακλάδωσης, όσο το δυνατόν περισσότερο.

Στην πραγματικότητα, χρειαζόμασταν μόνο ένα branch, για το loop με το οποίο κάναμε την προσπέλαση του πίνακα των 200 αριθμών. Εκμεταλλευόμενοι τη φύση του προβλήματος, αλλάξαμε τα branch με slt (set on less than) με τελεστές τον $zero και τον register του αριθμού που ελέγχουμε ($zero < αριθμός) για να υπολογίσουμε το άθροισμα των θετικών αριθμών και στη συνέχεια μία εντολή sltiu (set on less than immidiate unsigned) με τελεστές τον register του αριθμού που ελέγχουμε και την immidiate τιμή 1 (|αριθμός| < 1) για να υπολογίσουμε το άθροισμα των μηδενικών.
Μειώσαμε, κατ' αυτόν τον τρόπο και το πλήθος των συνολικών κύκλων για τον mipsB.
Ο mipsC και ο mipsA δεν είχανε καμία βελτίωση κατά την τροποποίηση αυτή καθώς:
Ο mipsA δεν έχει καθόλου hazards εφόσον δεν χρησιμοποιεί pipeline και
Ο mipsC έχει ελάχιστα control hazards λόγω της ύπαρξης του branch predictor και της επίλυσης των branches στο στάδιο ID (instruction decode).

```
    loop:
     lw $t7, 0($t0)                  # Load current element of A
     addi $t1, $t1, -1               # Decrement size counter
     addi $t0, $t0, 4                # Move to next element in A

     slt $t8,$zero,$t7    #1 if positive?

     sltiu $s0,$t7,1      #1 if less than one

     andi $t9, $t7, 1      # Perform bitwise AND with 1

     add $t3,$t3,$t8      #updating positive numbers

     add $t4, $t4, $s0    #adding zero?

     add $t6,$t6,$t9      #adding odd numbers
#loop check
     bne $t1, $zero, loop


#find negative and even sums
addi $t1,$zero,200
sub $t5,$t1,$t6 #even numbers

sub $t2,$t1,$t3
sub $t2,$t2,$t4#negative numbers are not zero and not positive
```

Η αλλαγή αυτή αποτέλεσε άλλο ένα milestone στην προσπάθειά μας για καλύτερες αποδόσεις :

MipsA

Cycle Statistics	
Total Cycles:	1955
Instructions:	1955
CPI:	1
Data Hazard Stalls:	0
Control Hazard Stalls:	0
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

MipsB

Cycle Statistics	
Total Cycles:	3309
Instructions:	1958
CPI:	1.68999
Data Hazard Stalls:	351
Control Hazard Stalls:	1000
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

mipsC

Cycle Statistics	
Total Cycles:	1994
Instructions:	1958
CPI:	1.01839
Data Hazard Stalls:	0
Control Hazard Stalls:	36
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

Επιλύσαμε για άλλη μία φορά όσα data hazards μπορούσαμε και καταλήξαμε σε ένα πρόγραμμα στο οποίο δεν μπορούσαμε να μειώσουμε περεταίρω τα control hazards και τα συνολικά instructions. Ο μόνος τρόπος να ελαττώσουμε κι άλλο τους συνολικούς κύκλους θα ήταν σε κάθε επανάληψη να εξετάζουμε παραπάνω στοιχεία του πίνακα, μειώνοντας έτσι τις συνολικές επαναλήψεις, τα control hazards που τις συνοδεύουν, καθώς και τα συνολικά instructions (μειώνουμε μία φορά τον counter του loop για παραπάνω αριθμούς).

Χρησιμοποιήσαμε, λοιπόν, την τεχνική του loop unrolling μέσω της οποίας υπήρξε σημαντική βελτίωση και στους 3 επεξεργαστές.

Παρακάτω παρατίθενται οι αποδόσεις του προγράμματος για κάθε επεξεργαστή με 8πλο loop unroll:

MipsA

Cycle Statistics	
Total Cycles:	1494
Instructions:	1494
CPI:	1
Data Hazard Stalls:	0
Control Hazard Stalls:	0
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

MipsB

Cycle Statistics	
Total Cycles:	1564
Instructions:	1497
CPI:	1.04476
Data Hazard Stalls:	17
Control Hazard Stalls:	50
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

MipsC

Cycle Statistics	
Total Cycles:	1500
Instructions:	1497
CPI:	1.002
Data Hazard Stalls:	0
Control Hazard Stalls:	3
RAM Stalls:	0
L1 Data Stalls:	0
L1 Program Stalls:	0
L2 Unified Stalls:	0

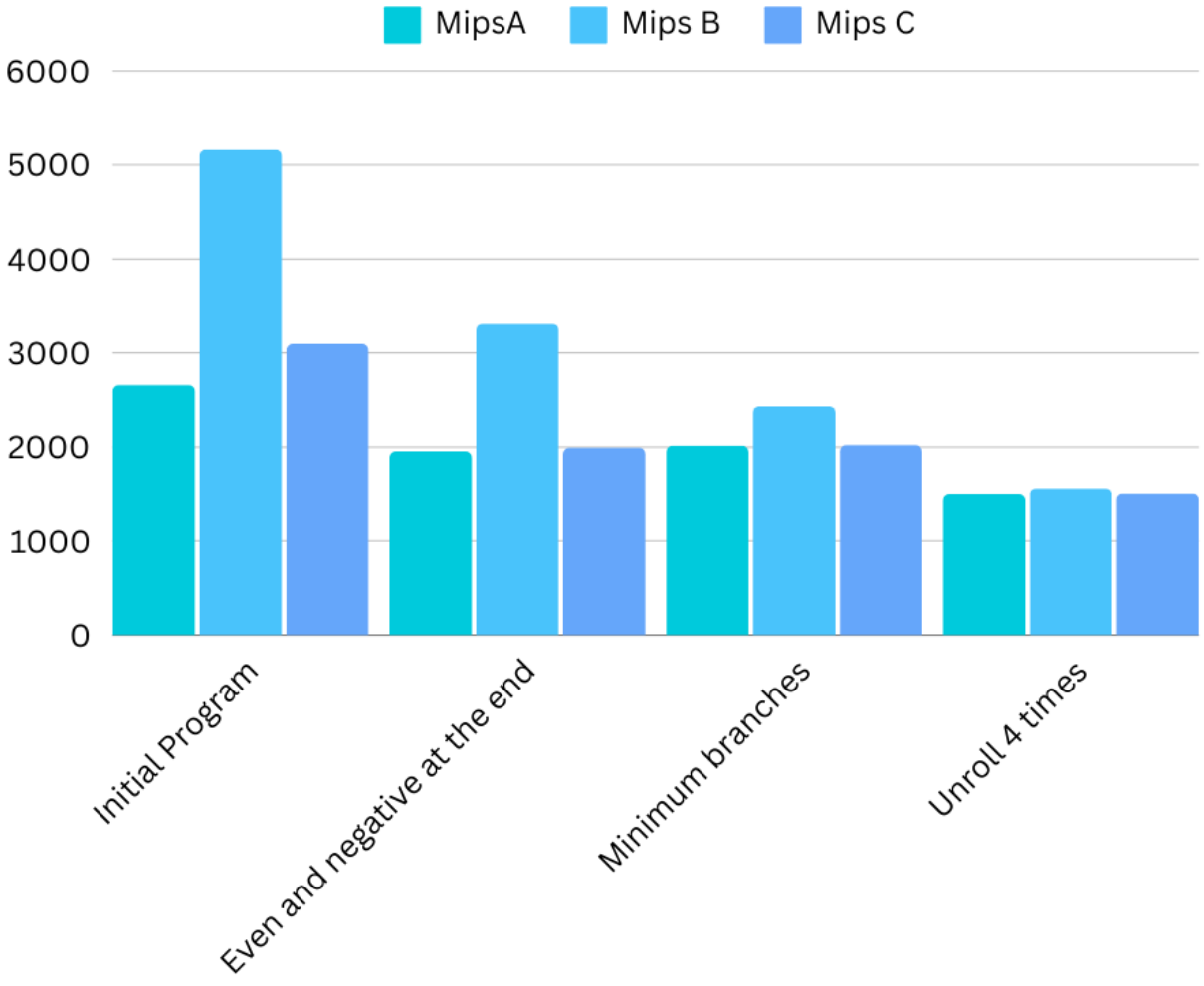
Ακολουθούν τα αποτελέσματα του τελικού μας προγράμματος final.s το οποίο έχει επισυναφθεί:

Registers

Decimal

$0/zero	0	$1/at	0	$2/v0	0	$3/v1	0	$4/a0	0
$5/a1	0	$6/a2	0	$7/a3	0	$8/t0	1000800	$9/t1	200
$10/t2	68	$11/t3	101	$12/t4	31	$13/t5	108	$14/t6	92
$15/t7	7	$16/s0	0	$17/s1	0	$18/s2	0	$19/s3	0
$20/s4	0	$21/s5	0	$22/s6	0	$23/s7	0	$24/t8	0
$25/t9	1	$26/k0	0	$27/k1	0	$28/gp	0	$29/sp	3221225216
$30/fp	0	$31/ra	0	pc	2147615032	lo	0	hi	0

Παρατίθεται σχεδιάγραμμα με τις επιδόσεις του κάθε επεξεργαστή σε κάθε milestone της εκπόνησης της εργασίας.



ΕΠΙΣΚΟΠΗΣΗ

Στο αρχικό πρόγραμμα (αρχείο start.s) πλήττεται από τα πολλά data hazards και κυρίως από τα ακόμα περισσότερα control hazads. Σταδιακά θεραπεύσαμε τις καθυστερήσεις (stalls) αλλά και μειώσαμε τις συνολικές εντολές προς εκτέλεση ανιχνεύοντας και εξαλείφοντας bottlenecks, αναδιατάσσοντας, πολλές φορές, τον κώδικα και εύρεσης ενός πιο αποδοτικού αλγορίθμου.

Ο mipsA είχε σημαντική βελτίωση στους συνολικούς κύκλους όταν μειώναμε τις συνολικές εντολές προς εκτέλεση στο πρόγραμμα.

Στον mipsB, φαίνεται ξεκάθαρα η δραματική βελτίωση της απόδοσης του προγράμματος σε κάθε στάδιο που θεωρήσαμε milestone (στάδια διαγράμματος). Μάλιστα, από το πρώτο στο τέταρτο στάδιο έχουμε βελτιωμένη κατά 330% απόδοση.

Όπως έχουμε ήδη αναφέρει είχε πάρα πολλά hazards, κυρίως

Ο mipsC είχε σημαντική βελτίωση στους συνολικούς κύκλους όταν μειώναμε τις συνολικές εντολές προς εκτέλεση στο πρόγραμμα και όταν μειώναμε τα control και data hazards, αλλά εφόσον δεν ήταν πολλά δεν επωφελήθηκε όσο ο mipsB...

Υπολογισμοί για την εύρεση του επεξεργαστή εκείνου που θα βελτιστοποιήσει (μεγιστοποιήσει) τον λόγο:

Υπολογιστική Απόδοση / Συνολικό Κόστος Ιδιοκτησίας

Υπολογιστική Απόδοση = Αριθμός CPU * το πλήθος των πλήρων εκτελέσεων του προγράμματος που εκτελούν όλες οι CPU σε ένα έτος

Συνολικό Κόστος Ιδιοκτησίας = κόστος αγοράς + δαπάνη ενέργειας για την ετήσια λειτουργία όλων των CPU

MipsA:

Υπολογιστική Απόδοση = 10.000 * 1.056.086.420 = 10,56 τρισεκατομμύρια εκτελέσεις

Συνολικό Κόστος Ιδιοκτησίας = 10.000 * 5euro + 200kW * 8.760ώρες * 0,002euro/kWh = 53.504 euro

Υπολογιστική Απόδοση / Συνολικό Κόστος Ιδιοκτησίας = 10,56τρις / 53.504

MipsB:

Υπολογιστική Απόδοση = 10.000 * 3.027.582.000 = 30,27 τρισεκατομμύρια εκτελέσεις

Συνολικό Κόστος Ιδιοκτησίας = 10.000 * 8 + 350kW * 8.760ώρες * 0,002euro/kWh = 86.132 euro

Υπολογιστική Απόδοση / Συνολικό Κόστος Ιδιοκτησίας = 30,27τρις / 86.132

MipsC:

Υπολογιστική Απόδοση = 10.000 * 3.153.600.000 = 31,53 τρισεκατομμύρια εκτελέσεις

Συνολικό Κόστος Ιδιοκτησίας = 10.000 * 12 + 450kW * 8.760ώρες * 0,002euro/kWh = 127.884 euro

Υπολογιστική Απόδοση / Συνολικό Κόστος Ιδιοκτησίας = 31,53τρις / 127.884

Επομένως το κλάσμα μεγιστοποιείται στον MipsB

Τεκμηρίωση

[Σύντομη τεκμηρίωση της λύσης σας **μέχρι 10 σελίδες ξεκινώντας από την επόμενη σελίδα** – μην αλλάζετε τη μορφοποίηση του κειμένου (**και παραδώστε την τεκμηρίωση σε αρχείο PDF**). Η τεκμηρίωσή σας πρέπει να περιλαμβάνει παραδείγματα ορθής εκτέλεσης του προγράμματος και σχολιασμό για την επίλυση του προβλήματος και την επίτευξη του ζητούμενου. Μπορείτε να χρησιμοποιήσετε εικόνες, διαγράμματα και ό,τι άλλο μπορεί να βοηθήσει στην εξήγηση της δουλειάς σας.]